

МИНИСТЕРСТВО ЛЕСНОГО ХОЗЯЙСТВА И ОХРАНЫ ОБЪЕКТОВ ЖИВОТНОГО МИРА
НИЖЕГОРОДСКОЙ ОБЛАСТИ

Государственное бюджетное профессиональное образовательное
учреждение Нижегородской области

«КРАСНОБАКОВСКИЙ ЛЕСНОЙ КОЛЛЕДЖ»

(ГБПОУ НО «КБЛК»)

День 15.

**«Разработка комплексных тестовых сценариев (Unit + Integration +
Contract)»**

Выполнил: Рахманов А. И.

Студент 24-ИС.

р.п. Красные Баки 2026г.

Введение

Цель

Разработать набор тестов (модульных, интеграционных и контрактных) для микросервиса «Система управления заказами и доставкой», обеспечивающих надежность модулей авторизации, обработки заказов и внешних интеграций (платежи, логистика).

Вариант и задание

Вариант 5. Параметризованные тесты для валидации

Задача: Проверка валидации DTO.

Условие: Есть OrderCreateDTO с полями phone (регулярка) и email.

Требование: Написать параметризованный тест (@pytest.mark.parametrize), который проверяет:

- 5 некорректных номеров (ожидается ошибка валидации).
- 5 корректных.

Структура

```
day15/
|
|— app/
|   |— __init__.py
|   |— schemas.py
|
|— tests/
|   |— __init__.py
|   |— conftest.py
|   |— test_schemas.py
|
|— .vscode/
|   |— settings.json
|
|— requirements.txt
|— pytest.ini
```

Код

Код программы

```
from pydantic import BaseModel, Field, EmailStr, field_validator
import re

class OrderCreateDTO(BaseModel):
    phone: str = Field(..., description="Номер телефона в формате +7XXX...")
    email: EmailStr = Field(..., description="Email адрес")

    @field_validator('phone')
    @classmethod
    def validate_phone(cls, value: str) -> str:
        """Валидация телефонного номера."""
        # Убираем все нецифровые символы, кроме +
        cleaned = re.sub(r'^\d+', '', value)

        # Проверяем, что номер начинается с +7 или 8 и содержит 11 цифр
        pattern = r'^(\+7|8)\d{10}$'
        if not re.match(pattern, cleaned):
            raise ValueError(f'Некорректный номер телефона: {value}')

        # Нормализуем: приводим к формату +7XXXXXXXXXX
        if cleaned.startswith('8'):
            cleaned = '+7' + cleaned[1:]

        return cleaned
```

Приложение 1. schemas

Код тестирования

```
"""
Тесты для валидации DTO (Вариант 5).
Проверяем валидацию телефонных номеров и email адресов.
"""

import pytest
import re
from pydantic import ValidationError
from app.schemas import OrderCreateDTO

# =====
# 1. ТЕСТЫ ВАЛИДАЦИИ ТЕЛЕФОННЫХ НОМЕРОВ
# =====

@pytest.mark.parametrize('phone,expected_valid', [
    # ---- КОРРЕКТНЫЕ номера (должны проходить валидацию) ----
    ('+79991234567', True),
    ('89991234567', True),
```

```

('+7-999-123-45-67', True),
('+7 999 123 45 67', True),
('+7(999)123-45-67', True),

# ---- НЕКОРРЕКТНЫЕ номера (должны выбрасывать ошибку) ----
('123', False),          # слишком короткий
('abc', False),          # буквы
('+7999123456', False),   # 10 цифр (не хватает)
('+799912345678', False), # 12 цифр (перебор)
('+1-999-123-45-67', False), # не Россия
])

def test_phone_validation(phone: str, expected_valid: bool):
    """
    Проверка валидации телефонных номеров.
    Ожидаем успех для корректных, ошибку для некорректных.

    AAA-сценарий:
    - ARRANGE: входные данные из параметризации
    - ACT: создание DTO или перехват исключения
    - ASSERT: проверка нормализации или наличия ошибки
    """

    # =====
    # ARRANGE (Подготовка) - данные из параметризации
    # =====
    email = 'test@example.com'

    # =====
    # ACT (Действие) - выполняем тестируемый код
    # =====
    if expected_valid:
        dto = OrderCreateDTO(phone=phone, email=email)
    else:
        with pytest.raises(ValidationError):
            OrderCreateDTO(phone=phone, email=email)
        return

    # =====
    # ASSERT (Проверка) - проверяем результат
    # =====
    if phone.startswith('8'):
        assert dto.phone == '+7' + phone[1:]
    else:
        expected = re.sub(r'^\d+', '', phone)
        assert dto.phone == expected

# =====
# 2. ТЕСТЫ ВАЛИДАЦИИ EMAIL
# =====

@pytest.mark.parametrize('email,expected_valid', [

```

```

# Корректные email
('user@example.com', True),
('admin@company.ru', True),
('test+filter@gmail.com', True),
('user.name@domain.org', True),
('info@sub.domain.com', True),

# Некорректные email
('not-an-email', False),
('missing@domain', False),
('@example.com', False),
('user@.com', False),
('user@domain', False),
('', False),
])

def test_email_validation(email: str, expected_valid: bool):
    """
    Проверка валидации email через EmailStr из Pydantic.

    AAA-сценарий:
    - ARRANGE: входные данные из параметризации
    - ACT: создание DTO или перехват исключения
    - ASSERT: проверка корректности email или наличия ошибки
    """

    # =====
    # ARRANGE (Подготовка)
    # =====
    phone = '+79991234567'

    # =====
    # ACT (Действие)
    # =====
    if expected_valid:
        dto = OrderCreateDTO(phone=phone, email=email)
    else:
        with pytest.raises(ValidationError):
            OrderCreateDTO(phone=phone, email=email)
        return

    # =====
    # ASSERT (Проверка)
    # =====
    assert dto.email == email

# =====
# 3. КОМПЛЕКСНЫЙ ТЕСТ (ПОЛНАЯ ВАЛИДАЦИЯ)
# =====

def test_order_create_dto_full_validation():
    """

```

Проверка, что DTO создаётся только с полностью корректными данными.

AAA-сценарий:

- ARRANGE: подготовка валидных и невалидных данных
- ACT: создание DTO в 3 разных сценариях
- ASSERT: проверка всех результатов

"""

```
# =====
# ARRANGE (Подготовка)
# =====

valid_phone = '+79991234567'
valid_email = 'user@example.com'
invalid_phone = '123'
invalid_email = 'bad-email'

# =====
# ACT (Действие)
# =====

# Сценарий 1: Валидные данные
dto = OrderCreateDTO(phone=valid_phone, email=valid_email)

# Сценарий 2: Невалидный телефон
with pytest.raises(ValidationError) as exc1:
    OrderCreateDTO(phone=invalid_phone, email=valid_email)

# Сценарий 3: Невалидный email
with pytest.raises(ValidationError) as exc2:
    OrderCreateDTO(phone=valid_phone, email=invalid_email)

# =====
# ASSERT (Проверка)
# =====

assert dto.phone == valid_phone
assert dto.email == valid_email
assert 'Некорректный номер телефона' in str(exc1.value)
assert 'value_error' in str(exc2.value).lower() or 'validation' in
str(exc2.value).lower()
```

```
# =====
# 4. ТЕСТ НА СООБЩЕНИЕ ОБ ОШИБКЕ
# =====
```

```
def test_phone_error_message():
    """
```

Проверка, что сообщение об ошибке содержит понятный текст.

AAA-сценарий:

- ARRANGE: некорректный телефон
- ACT: перехват исключения
- ASSERT: проверка текста ошибки

```

"""
# =====
# ARRANGE (Подготовка)
# =====
invalid_phone = '+7999123456' # не хватает одной цифры
email = 'test@example.com'

# =====
# АКТ (Действие)
# =====
with pytest.raises(ValidationError) as exc_info:
    OrderCreateDTO(phone=invalid_phone, email=email)

# =====
# ASSERT (Проверка)
# =====
error_message = str(exc_info.value)
assert 'Некорректный номер телефона' in error_message

# =====
# 5. ТЕСТ НА ГРАНИЧНЫЕ СЛУЧАИ
# =====

@pytest.mark.parametrize('phone', [
    '', # пустая строка
    '+', # только плюс
    '79991234567', # без +7 или 8
    '+7abc', # буквы
    '8' * 15, # слишком длинный
    '8' * 10, # 10 цифр вместо 11
    '+7' + '1' * 20, # слишком длинный с +7
])
def test_phone_edge_cases(phone: str):
    """
    Проверка граничных случаев для телефона.

    AAA-сценарий:
    - ARRANGE: граничные значения из параметризации
    - АКТ + ASSERT: проверка, что все выбрасывают ошибку
    """
    # =====
    # ARRANGE (Подготовка) - данные из параметризации
    # =====
    email = 'test@example.com'

    # =====
    # АКТ (Действие) + ASSERT (Проверка)
    # =====
    with pytest.raises(ValidationError):
        OrderCreateDTO(phone=phone, email=email)

```

```

# =====
# 6. ТЕСТ С ИСПОЛЬЗОВАНИЕМ ФИКСТУР
# =====

def test_with_fixtures(valid_order_data, valid_dto, sample_phones,
sample_emails):
    """
    Тест с использованием фикстур для демонстрации.

    AAA-сценарий:
    - ARRANGE: данные из фикстур
    - ACT: проверка валидных данных
    - ASSERT: все проверки
    """

    # =====
    # ARRANGE (Подготовка) - данные из фикстур
    # =====
    # valid_order_data, valid_dto, sample_phones, sample_emails уже готовы

    # =====
    # ACT (Действие) - проверяем валидные данные
    # =====
    for phone in sample_phones['valid']:
        dto = OrderCreateDTO(phone=phone, email='test@example.com')
        # Проверяем, что DTO создаётся для всех валидных телефонов

    for email in sample_emails['valid']:
        dto = OrderCreateDTO(phone='+79991234567', email=email)
        # Проверяем, что DTO создаётся для всех валидных email

    # =====
    # ASSERT (Проверка)
    # =====
    # Проверяем, что валидные данные из фикстур корректны
    assert valid_dto.phone == '+79991234567'
    assert valid_dto.email == 'test@example.com'

    assert len(sample_phones['valid']) == 5
    assert len(sample_phones['invalid']) == 5
    assert len(sample_emails['valid']) == 3
    assert len(sample_emails['invalid']) == 4

```

Приложение 2. test_schemas

```

"""
Глобальные фикстуры и настройки для всех тестов.
"""

import sys
from pathlib import Path

```



```

# Добавляем корень проекта в PYTHONPATH
root_dir = Path(__file__).parent.parent
if str(root_dir) not in sys.path:
    sys.path.insert(0, str(root_dir))

import pytest
from app.schemas import OrderCreateDTO

@pytest.fixture
def valid_order_data():
    """Фикстура с валидными данными для заказа."""
    return {
        "phone": "+79991234567",
        "email": "test@example.com"
    }

@pytest.fixture
def valid_dto(valid_order_data):
    """Фикстура с созданным валидным DTO."""
    return OrderCreateDTO(**valid_order_data)

@pytest.fixture
def sample_phones():
    """Фикстура с примерами телефонов для тестирования."""
    return {
        "valid": [
            "+79991234567",
            "89991234567",
            "+7-999-123-45-67",
            "+7 999 123 45 67",
            "+7(999)123-45-67"
        ],
        "invalid": [
            "123",
            "abc",
            "+7999123456",
            "+799912345678",
            "+1-999-123-45-67"
        ]
    }

@pytest.fixture
def sample_emails():
    """Фикстура с примерами email для тестирования."""
    return {
        "valid": [
            "user@example.com",

```

```
        "admin@company.ru",  
        "test+filter@gmail.com"  
    ],  
    "invalid": [  
        "not-an-email",  
        "missing@domain",  
        "@example.com",  
        "user@.com"  
    ]  
}
```

Приложение 3. config

Вывод

В ходе выполнения практической работы был разработан комплексный набор параметризованных тестов для проверки валидации DTO OrderCreateDTO с использованием Pydantic V2 и pytest. Создана полноценная структура проекта с разделением на бизнес-логику (app/schemas.py) и тесты (tests/test_schemas.py), настроено тестовое окружение с виртуальным окружением и всеми необходимыми зависимостями. Реализована валидация телефонных номеров с поддержкой различных форматов (включая +7, 8, дефисы, скобки и пробелы) и их нормализация к единому формату +7XXXXXXXXXX, а также валидация email-адресов через встроенный тип EmailStr. Тесты покрывают 5 корректных и 5 некорректных вариантов телефонов, 5 валидных и 5 невалидных email-адресов, граничные случаи (пустая строка, спецсимволы, слишком длинные номера) и проверку текста сообщений об ошибках. Все тесты оформлены по паттерну AAA (Arrange-Act-Assert), используют параметризацию через @pytest.mark.parametrize и фикстуры из `conftest.py` для переиспользования данных. В результате все 31 тест успешно проходят, покрытие кода составляет 100%, предупреждения отсутствуют, что подтверждает корректность работы валидации и надёжность реализованного функционала. Задание выполнено в полном объёме, работа готова к сдаче.

Проведение тестов

```
Командная строка

===== 31 passed in 0.52s =====

Y:\24 ИСИП 2025\УП02\Группа 2\Рахманов А\day15>pytest tests/test_schemas.py -v
===== test session starts =====
platform win32 -- Python 3.13.9, pytest-9.1.0, pluggy-1.6.0 -- C:\Users\Student240\AppData\Local\Programs\Python\Python3
13\python.exe
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: Y:\24 ИСИП 2025\УП02\Группа 2\Рахманов А\day15
configfile: pytest.ini
plugins: hypothesis-6.155.3, cov-7.1.0, gremlins-1.8.1
collected 31 items

tests/test_schemas.py::test_phone_validation[+79991234567-True] PASSED [ 3%]
tests/test_schemas.py::test_phone_validation[89991234567-True] PASSED [ 6%]
tests/test_schemas.py::test_phone_validation[+7-999-123-45-67-True] PASSED [ 9%]
tests/test_schemas.py::test_phone_validation[+7 999 123 45 67-True] PASSED [ 12%]
tests/test_schemas.py::test_phone_validation[+7(999)123-45-67-True] PASSED [ 16%]
tests/test_schemas.py::test_phone_validation[123-False] PASSED [ 19%]
tests/test_schemas.py::test_phone_validation[abc-False] PASSED [ 22%]
tests/test_schemas.py::test_phone_validation[+7999123456-False] PASSED [ 25%]
tests/test_schemas.py::test_phone_validation[+799912345678-False] PASSED [ 29%]
tests/test_schemas.py::test_phone_validation[+1-999-123-45-67-False] PASSED [ 32%]
tests/test_schemas.py::test_email_validation[user@example.com-True] PASSED [ 35%]
tests/test_schemas.py::test_email_validation[admin@company.ru-True] PASSED [ 38%]
tests/test_schemas.py::test_email_validation[test+filter@gmail.com-True] PASSED [ 41%]
tests/test_schemas.py::test_email_validation[user.name@domain.org-True] PASSED [ 45%]
tests/test_schemas.py::test_email_validation[info@sub.domain.com-True] PASSED [ 48%]
tests/test_schemas.py::test_email_validation[not-an-email-False] PASSED [ 51%]
tests/test_schemas.py::test_email_validation[missing@domain-False] PASSED [ 54%]
tests/test_schemas.py::test_email_validation[@example.com-False] PASSED [ 58%]
tests/test_schemas.py::test_email_validation[user@.com-False] PASSED [ 61%]
tests/test_schemas.py::test_email_validation[user@domain-False] PASSED [ 64%]
tests/test_schemas.py::test_email_validation[-False] PASSED [ 67%]
tests/test_schemas.py::test_order_create_dto_full_validation PASSED [ 70%]
tests/test_schemas.py::test_phone_error_message PASSED [ 74%]
tests/test_schemas.py::test_phone_edge_cases[] PASSED [ 77%]
tests/test_schemas.py::test_phone_edge_cases[+] PASSED [ 80%]
tests/test_schemas.py::test_phone_edge_cases[79991234567] PASSED [ 83%]
tests/test_schemas.py::test_phone_edge_cases[+7abc] PASSED [ 87%]
tests/test_schemas.py::test_phone_edge_cases[8888888888888888] PASSED [ 90%]
tests/test_schemas.py::test_phone_edge_cases[88888888888] PASSED [ 93%]
tests/test_schemas.py::test_phone_edge_cases[+7111111111111111111] PASSED [ 96%]
tests/test_schemas.py::test_with_fixtures PASSED [100%]

===== 31 passed in 0.51s =====
```

Приложение 4. Проведение тестов

```
Командная строка

plugins: hypothesis-6.155.3, cov-7.1.0, gremlins-1.8.1
collected 31 items

tests/test_schemas.py::test_phone_validation[+79991234567-True] PASSED [ 3%]
tests/test_schemas.py::test_phone_validation[89991234567-True] PASSED [ 6%]
tests/test_schemas.py::test_phone_validation[+7-999-123-45-67-True] PASSED [ 9%]
tests/test_schemas.py::test_phone_validation[+7 999 123 45 67-True] PASSED [ 12%]
tests/test_schemas.py::test_phone_validation[+7(999)123-45-67-True] PASSED [ 16%]
tests/test_schemas.py::test_phone_validation[123-False] PASSED [ 19%]
tests/test_schemas.py::test_phone_validation[abc-False] PASSED [ 22%]
tests/test_schemas.py::test_phone_validation[+7999123456-False] PASSED [ 25%]
tests/test_schemas.py::test_phone_validation[+799912345678-False] PASSED [ 29%]
tests/test_schemas.py::test_phone_validation[+1-999-123-45-67-False] PASSED [ 32%]
tests/test_schemas.py::test_email_validation[user@example.com-True] PASSED [ 35%]
tests/test_schemas.py::test_email_validation[admin@company.ru-True] PASSED [ 38%]
tests/test_schemas.py::test_email_validation[test+filter@gmail.com-True] PASSED [ 41%]
tests/test_schemas.py::test_email_validation[user.name@domain.org-True] PASSED [ 45%]
tests/test_schemas.py::test_email_validation[info@sub.domain.com-True] PASSED [ 48%]
tests/test_schemas.py::test_email_validation[not-an-email-False] PASSED [ 51%]
tests/test_schemas.py::test_email_validation[missing@domain-False] PASSED [ 54%]
tests/test_schemas.py::test_email_validation[@example.com-False] PASSED [ 58%]
tests/test_schemas.py::test_email_validation[user@.com-False] PASSED [ 61%]
tests/test_schemas.py::test_email_validation[user@domain-False] PASSED [ 64%]
tests/test_schemas.py::test_email_validation[-False] PASSED [ 67%]
tests/test_schemas.py::test_order_create_dto_full_validation PASSED [ 70%]
tests/test_schemas.py::test_phone_error_message PASSED [ 74%]
tests/test_schemas.py::test_phone_edge_cases[] PASSED [ 77%]
tests/test_schemas.py::test_phone_edge_cases[+] PASSED [ 80%]
tests/test_schemas.py::test_phone_edge_cases[79991234567] PASSED [ 83%]
tests/test_schemas.py::test_phone_edge_cases[+7abc] PASSED [ 87%]
tests/test_schemas.py::test_phone_edge_cases[8888888888888888] PASSED [ 90%]
tests/test_schemas.py::test_phone_edge_cases[88888888888] PASSED [ 93%]
tests/test_schemas.py::test_phone_edge_cases[+7111111111111111111] PASSED [ 96%]
tests/test_schemas.py::test_with_fixtures PASSED [100%]

===== tests coverage =====
_____ coverage: platform win32, python 3.13.9-final-0 _____

Name          Stmts  Miss  Cover   Missing
-----
app\__init__.py      0      0  100%
app\schemas.py     15      0  100%
-----
TOTAL              15      0  100%

===== 31 passed in 0.90s =====

Y:\24 ИСИП 2025\УП02\Группа 2\Рахманов А\day15>
```

Приложение 5. Покрытие тестов